

Multi-Agent Large Language Models with AutoGen

Janik Jehkul , Nikita Kostin , and Xiangyu Ning 

TUM School of Computation, Information and Technology, Technical University of Munich

 janik.jehkul@tum.de

 nikita.kostin@tum.de

 xiangyu.ning@tum.de

August 17, 2024

Abstract —

With the ever-increasing interest in large language models (LLMs), the range of possible tasks where LLMs can be applied grows. Empirical studies show that multiple LLM agents participating in a conversation provide an improvement over a traditional single-agent approach. In our study, we explore various use cases where multi-agent LLMs provide a noticeable improvement. Furthermore, we implement an overarching agent that combines and manages the agent participants, aiming to apply to a wider range of tasks. Finally, we introduce a benchmark using the LLM-as-Evaluator approach and using it showcase the performance of the overarching agent.

1 Introduction

Recent advancements in large language models (LLMs) have demonstrated unprecedented capabilities in natural language understanding and generation across a wide range of tasks (Brown et al., 2020). These models excel in many areas, yet they face limitations when confronted with complex, multi-domain problems. Consider a scenario where a user needs to solve a complex problem that involves web research, code generation and image analysis. A single artificial intelligence (AI) agent might struggle to perform well across all these diverse areas. To address this challenge, recent work has focused on multi-agent systems that distribute sub-tasks among specialized large language models. This approach is similar to the collaborative strategies employed in human interdisciplinary problem-solving. The AutoGen framework (Wu et al., 2023) provides a flexible way to build these multi-agent systems. It aims to handle complex tasks better than single AI models by leveraging the strengths of different specialized agents.

Our study explores the effectiveness of AutoGen for developing an overarching agent capable of delegating tasks to specialized sub-agents across diverse domains.

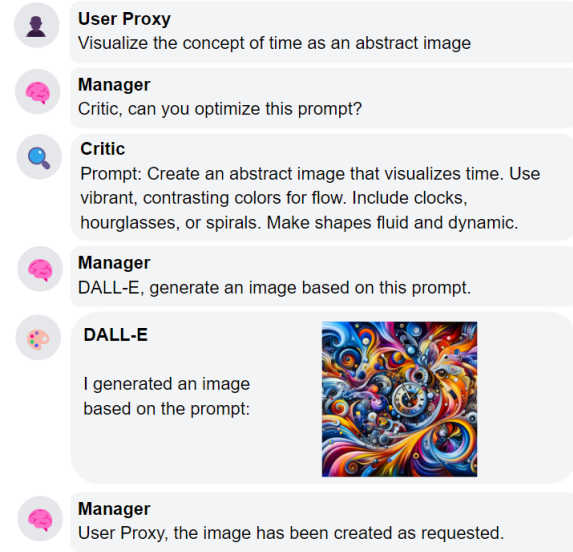


Figure 1 An example of a multi-agent chat

Specifically, we will explore the following key research questions (RQ):

- RQ1 Are the experiments conducted by the AutoGen authors reproducible?
- RQ2 How effectively can an overarching agent, implemented using the AutoGen framework, delegate tasks to specialized sub-agents and solve complex problems across diverse domains?
- RQ3 What are the possible scenarios and tasks that could benefit from a multi-agent approach?
- RQ4 What are the key considerations in designing user interfaces to facilitate the creation and utilization of multi-agent systems?

In this paper, we present the results and the findings of the study. In section 2, we discuss the related work of the AutoGen framework and the general multi-agent LLMs methodologies. In section 3, we describe our methodology for implementing and evaluating the overarching agent using the AutoGen framework. In section 4, we introduce relevant agent task benchmarks and contribute a novel SelectBenchmark to evaluate

the delegation capabilities of an overarching agent. In section 5, we replicate the original AutoGen findings and perform novel experiments demonstrating AutoGen’s capabilities in various domains. In section 6, we go over the results of the experiments from Section 5. In section 7 and section 8, we summarize our findings and point out the potential research directions.

2 Related Work

Multi-Agent LLM Systems Multiple modifications of the traditional single-agent LLM approach have been proposed lately. CAMEL (Li et al., 2023b) introduces a framework for autonomous cooperation between AI agents, using role-playing and inception prompting to guide conversations towards task completion. AutoGen (Wu et al., 2023) introduces a flexible framework to build multi-agent LLM applications. The authors of the paper showcase their approach on various scenarios, where each agent is dedicated to a certain portion of a task. The idea of AutoGen is further researched in (Zhang et al., 2024), where agents have access to functions, which in turn are treated as learnable parameters.

Reasoning and Task Planning Several approaches have been proposed to enhance the reasoning and task-planning capabilities of large language models. Chain-of-thought prompting (Wei et al., 2022) introduced the idea of including intermediate reasoning steps in prompts to improve performance on complex tasks. Building on this, ReAct (Yao et al., 2022) combined step-by-step reasoning with the ability to perform actions, allowing models to interact with their environment while solving problems. Tree-of-thoughts (Yao et al., 2023) extended the chain-of-thought ideas by modeling the reasoning process as a tree, allowing for exploration of multiple reasoning paths and backtracking. Graph-of-thoughts (Besta et al., 2024) proposed a more flexible framework that models reasoning as an arbitrary graph structure instead of a tree.

User Interfaces for Multi-Agent Systems To address the goal of making multi-agent systems easier to create, it is crucial to consider user interface developments. AutoGen Studio (Dibia et al., 2024), for instance, provides a graphical interface for designing and testing multi-agent systems built with AutoGen.

3 Methodology

An *agent* is an entity acting on behalf of human intent, capable of conversation and interaction with other agents. A *multi-agent conversation* is an abstraction of a chat where agents reason about a user’s request and try to come up with a suitable solution. An agent can contain a set of *tools*, which is a set of predefined functions, aimed to provide specific capabilities or actions, such as retrieving information, performing calculations, or generating content (Wu et al., 2023).

We use a concept of an *overarching agent* to manage the conversation in a multi-agent chat. An overarching agent is a parent agent that performs the following three functions:

1. Selection of the agents to participate in the discussion from an available agents list.
2. Selection of order for the agents to participate in.
3. Prompt managing of an initial user request.

The list of participating agents and their order is decided by the LLM itself and is managed by the AutoGen framework. The prompt management has a purpose of tying all of the available information together in a text message: e.g. if an image is provided, the overarching agent will mention this explicitly and pass the image path. As the goal of the overarching agent is to delegate the task, the meaning and formulation of the original request remain intact.

In Figure 2 the overarching agent is represented by *User Proxy* and *Manager*. The *User Proxy* agent is a pass-through agent that serves as an entry point for a user request. The *Manager* agent performs the above-mentioned assignment and prompts modifications. In the case of an online search request with a given image, the *Manager* will first call the agent *Image Explainer* to get the information, then pass it to the next agent *Web Retriever*.

The list of available chat agents by default contains Calculator, Web Retriever, Coding, Code Safeguard, Image Explainer, DALL-E, Critic and PDF Parser. These agents are predefined and each contains a set of tools targeted to solve a specific task. Further agents can be added manually by the user: these agents are represented as *Custom Agents* in Figure 2. The functionality of individual predefined agents is described in detail in section 5.

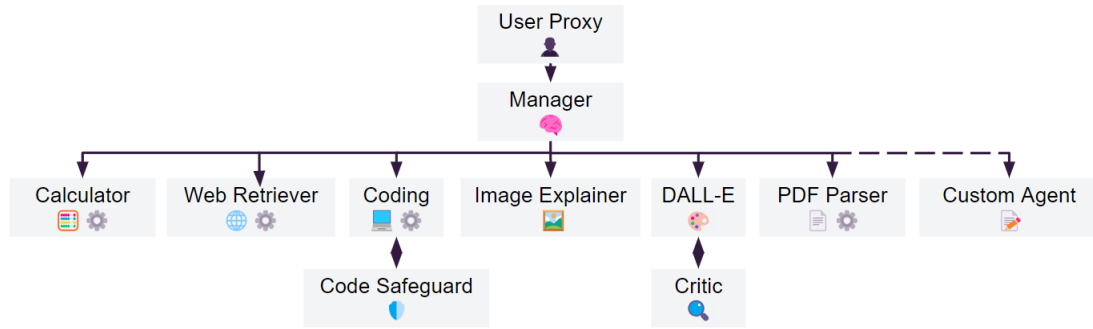


Figure 2 This figure illustrates an overarching agent architecture, showing a hierarchical structure with a *User Proxy* agent at the top passing queries to a *Manager* agent. The *Manager* agent dynamically selects from specialized agent subgroups for different tasks, including *Calculator*, *Web Retriever*, *Coding* (with *Code Safeguard*), *Image Explainer*, *DALL-E* (with *Critic*), *PDF Parser* or a *Custom Agent*, each represented by a relevant icon

4 Datasets

MATH The MATH dataset contains 7,500 training and 5,000 test samples of competition-level mathematics problems. It covers seven subjects including Prealgebra, Algebra, Number Theory, Counting and Probability, Geometry, Intermediate Algebra and Pre-calculus. Problems are categorized into five difficulty levels and include step-by-step solutions (Hendrycks et al., 2021).

Natural Questions Dataset The Natural Questions dataset (Kwiatkowski et al., 2019) is a large-scale question-answering corpus. It contains 307,373 training examples with single annotations and 15,672 examples with annotations from five different annotators (7,830 for development and 7,842 for testing). Each example pairs a real user query with a Wikipedia page that contains the related answer.

ALFWorld ALFWorld (Shridhar et al., 2021) is a dataset that aligns interactive text-based environments from TextWorld (Côté et al., 2018) with 3D embodied environments from the ALFRED dataset (Shridhar et al., 2020). It covers six types of household tasks, such as "Heat & Place" (e.g. "heat a mug and put it on the coffee table"). Each task requires completing a specific sequence of actions in a textual environment to succeed, like finding the object, heating it and placing it correctly.

OptiGuide OptiGuide (Li et al., 2023a) introduces an evaluation benchmark designed to assess the effectiveness of LLMs in interpreting and generating code for complex supply chain optimization problems.

MiniWoB++ Mini World of Bits (MiniWoB) introduced a benchmark of 100 synthetic web environments designed to evaluate AI agents on web interaction tasks. These environments present small (160x210 pixel) web interfaces where agents must interpret instructions and interact using mouse and keyboard actions (Shi et al., 2017). MiniWoB++ extends this concept with a new set of tasks including longer time horizons, "soft" reasoning about natural language and stochastically varying layouts. For example, tasks may involve forms with randomly ordered fields or dynamically changing UI layouts, rather than fixed interfaces (Liu et al., 2018).

SelectBenchmark We contribute a novel benchmark for evaluating multi-agent systems, which assesses the ability of an overarching agent to correctly route tasks to appropriate specialized agents and evaluates if the task was completed successfully. It consists of 120 diverse tasks across 6 categories (calculator, coding, DALL-E, image retriever, pdf parser and web retriever), with 20 tasks per category. Each task is structured with a unique ID, name, input prompt, expected agent and success criteria. The tasks were designed to be as unambiguous as possible, ensuring that only one of the six specialized agents should be called for a given task. To measure performance we employ three main performance metrics: only correct agent (evaluates whether the overarching agent exclusively called the most appropriate agent for the task), correct agent (checks if the correct agent was called (even if others were also called) and success ratio (measures overall performance). The benchmark uses an LLM-as-evaluator approach for automated and scalable assessment. The LLM-as-evaluator typically leverages

the textual response of the selected agent for assessment. However, since certain agents require specific modalities, some tasks provide additional context such as a PDF document for the PDF parser and an image for the image retriever. For tasks involving DALL-E, we utilize GPT-4 Vision to analyze the generated image and compare its textual description against the expected outcome. For a more comprehensive understanding of the benchmark’s implementation, we provide examples of task structures for each specialized agent in Tables 3-8 in the Appendix.

5 Experiments

We first reproduce the experiments as described in AutoGen (Wu et al., 2023) in subsection 5.1. Moreover, we introduce further tasks to showcase the AutoGen framework (subsection 5.2) and utilize our proposed SelectBenchmark to show the performance of the overarching agent on them (subsection 5.3).

5.1 Original Paper’s Findings

1. Math Problem Solving We replicated the math problem-solving experiments from AutoGen using the MATH dataset (Hendrycks et al., 2021). This experiment employed an AutoGen two-agent system, consisting of an assistant agent for step-by-step problem-solving and a user proxy agent. We evaluated the system’s performance using the success rate on 120 Level-5 problem-solving tasks, which represent the most challenging problems in the dataset.

2. RAG To replicate the original paper’s experiment on RAG we used the NaturalQuestions dataset (Kwiatkowski et al., 2019). The experiment involved a retrieve assistant agent and a retrieve user proxy agent, responsible for providing relevant chunks of the document as context. The experiment was performed with and without *interactive retrieval*: whenever the agent was not able to find the required context, it would request an updated context instead of terminating. Due to the large size of the dataset, to reduce the time and budget spent for the experiment, we executed the benchmark on the first 100 questions from a random shuffled dataset.

3. ALFWorld Text World Interaction We reproduced the ALFWorld experiments using the ALFWorld benchmark (Shridhar et al., 2021). This experiment involved both two-agent (action planner and

executor) and three-agent (adding a grounding agent that learns from failed attempts) AutoGen systems. We measured the task completion rate across 134 unseen tasks, reporting both average performance and the best result out of three attempts.

4. Multi-agent Coding To set up the multi-agent coding experiment, the authors of the paper use a critic agent and a writer agent. We replicated the results of the paper, compared it with the ChatGPT+Code Interpreter and showed the reductions in user time and user interactions.

5. Dynamic Group Chat To evaluate AutoGen’s capability in managing multi-agent conversations, we used the 12 manually crafted complex tasks from the original AutoGen paper (Wu et al., 2023). This experiment involved an AutoGen two-agent system with an assistant and a user proxy agent and a dynamic group chat system with multiple specialized agents (user proxy, engineer, critic, executor). The primary metric for this experiment was the success rate on task completion.

6. Conversational Chess We reproduced the findings of the original AutoGen paper (Wu et al., 2023) on the ability of agents to play complex board games, such as chess. Due to lack of a specific dataset the results of an experiment are empirical and were performed in three settings: with and without an intermediate board agent controlling the correctness of a move and using function calls. Regardless of the setup, two other "player" agents participated in the experiment. A visualization of a sample chess interaction can be found in Figure 5 in Appendix A.

7. MiniWoB++ Web Interaction We conducted experiments using the MiniWoB++ benchmark (Liu et al., 2018) to assess AutoGen’s performance in web-based tasks. This involved an AutoGen two-agent system consisting of an assistant agent for task analysis and an executor agent for carrying out web actions. The experiments utilized Kim et al. (2023) frameworks with different base prompts and in-context learning examples for each task. For instance, tasks like "click-checkboxes large" and "navigate tree" each had their own customized prompts and example plans. These examples provided step-by-step instructions for the autonomous agent, such as clicking specific checkboxes or navigating through file structure. To reduce API costs, we measured the average success rate out of 10

trials across 23 of the 47 available tasks in the benchmark

5.2 Further Experiments

To further demonstrate the capabilities of the AutoGen framework, we introduced further experiment setups. Their performance was evaluated using SelectBenchmark described in subsection 5.3.

1. PDFTriage Similar to the approach described in (Saad-Falcon et al., 2023), we implemented a PDF parsing agent using AutoGen. The agent receives a modified user prompt, containing the original question and a path to the PDF document and uses vector indexing to retrieve an answer from the PDF. The idea of the original paper was to reduce the amount of content available to the agent by retrieving only the relevant chunks of the document. We follow a similar approach and allow the agent to call functions for page fetching, section fetching and general retrieval. For a visual representation of the PDFTriage process, see Figure 6 in Appendix A.

2. DALL-E We integrated a DALL-E 3 (OpenAI, 2023) two-agent conversation using the AutoGen framework to enable image generation capabilities for our overarching agent. This integration leverages a critic agent, which analyzes the user’s initial text prompt and suggests improvement based on text clarity. This iterative process continues until the critic agent deems the prompt satisfactory and sends it to the DALL-E agent to generate an image. An example output from our DALL-E integration can be found in Figure 7 in Appendix A.

3. Vision To allow images as input for our overarching agent, we implemented an agent that is able to process visual information. The modified prompt points to the input file that is then passed to AutoGen, which in turn makes a corresponding API call. A visualization is provided in Figure 8 in Appendix A.

4. Interactive Browsing To enable the overarching agent to retrieve unknown information from the Internet, we implemented an agent with browser functionality. The browser agent, upon receiving a user request, decides whether it can answer the question without further context or whether it needs additional information. In case more information is needed, the agent can:

1. Make a Google API request to retrieve the first 10 Google search results, containing the title, a short text snippet and a link to the full result.
2. Using Selenium, fetch a specific URL to retrieve the contents of a page.

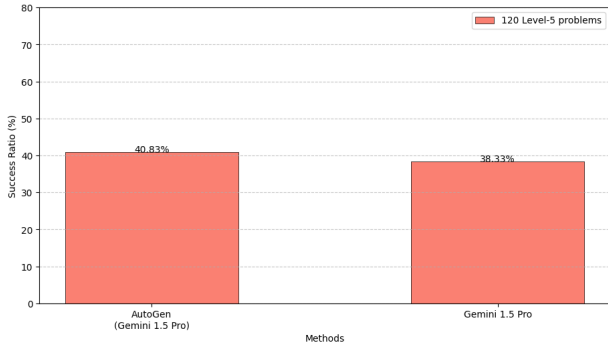
The retrieved information is used as context to answer the question. Figure 9 in Appendix A demonstrates the web search and retrieval process of our interactive browsing agent.

5.3 SelectBenchmark

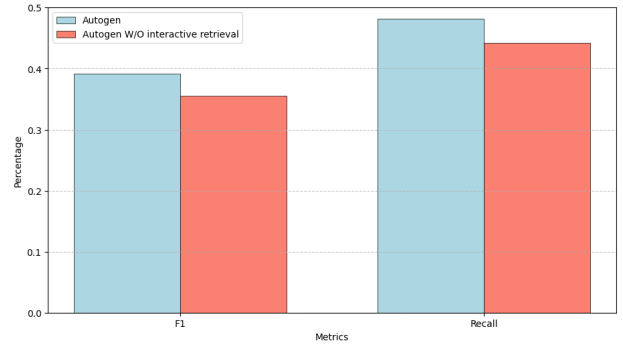
To evaluate AutoGen’s ability to select the correct child agent for a given task, we developed SelectBenchmark. This experiment utilizes the AutoGen multi-agent system with an overarching agent. Figure 2 illustrates the overarching agent and all utilized child agents.

We already described PDFTriage, DALL-E, Vision and Interactive Browsing agents in the previous paragraphs of 5.2. Additionally, we implemented a calculator agent for mathematical operations and a coding agent for writing and executing Python code. The calculator agent is designed to perform various mathematical operations. It can handle basic arithmetic (addition, subtraction, multiplication, division) as well as more complex operations like exponentiation, logarithms and finding the greatest common divisor or least common multiple of two numbers. The coding agent system consists of two collaborative agents: a coding agent that writes and explains Python code to solve programming problems and a safeguard agent that reviews the code for safety. The safeguard agent checks the code for potentially harmful operations, such as inappropriate file system access, unauthorized network connections, dangerous system calls or resource-intensive loops before the code can be executed.

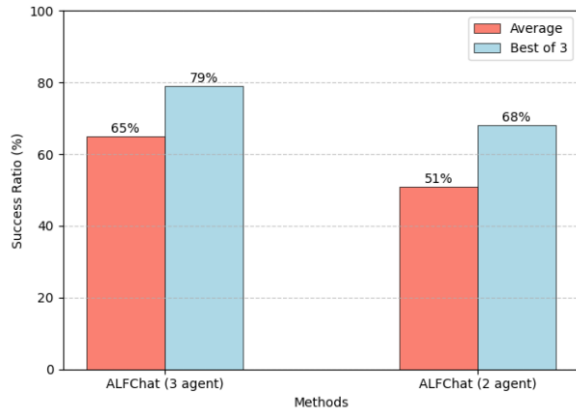
The performance of the overarching agent architecture on the SelectBenchmark is measured by whether the correct agent was called (even if others were also called), whether only the correct agent was called and the overall task success ratio. Examples of tasks and their corresponding specialized agent can be found in Tables 3-8 in the Appendix A.



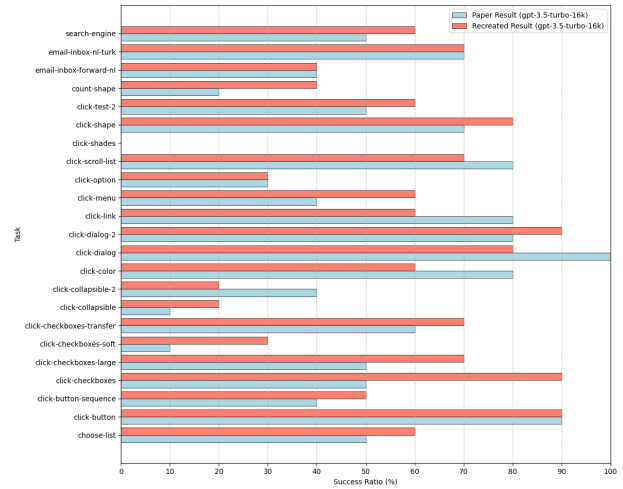
(a) Performance on MATH



(b) Performance on Natural Question Dataset



(c) ALFWorld Text World Interaction



(d) MiniWoB++ Web Interaction: Performance on 23 out of 47 tasks

Figure 3 (a) Performance evaluation on the MATH dataset (b) Performance on the Natural Question Dataset (c) Results from ALFWorld Text World Interaction benchmark (d) Performance metrics on MiniWoB++ Web Interaction tasks

6 Results

This section contains the results of the reproduced experiments taken from the original paper, as well as the performance of our overarching agent on the SelectBenchmark.

6.1 Math Problem Solving

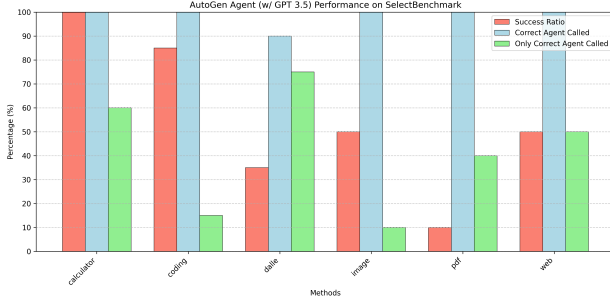
Figure 3a shows the performance on math problem-solving. AutoGen with Gemini 1.5 Pro achieved the highest success rate at 40.83% on 120 Level-5 problems. Gemini 1.5 Pro alone performed slightly lower at 38.33%. Compared to the original paper AutoGen’s performance decreased from 52.5% to 40.83% on 120 Level-5 problems when using Gemini 1.5 Pro but in both cases, AutoGen two-agent system achieved the highest performance on this respective benchmark.

6.2 RAG

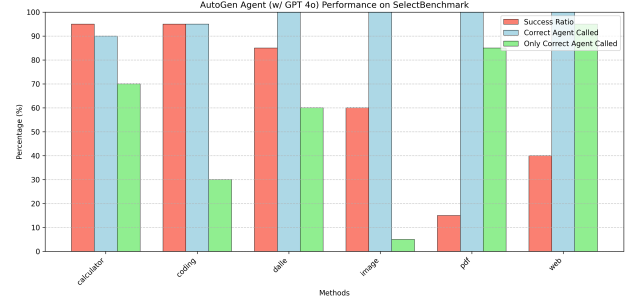
Figure 3b contains the results of the RAG evaluation. Similar to the original paper’s findings, interactive retrieval provided a boost in performance, achieving 39.2% in F1 and 48.18% in Recall, as compared to 35.59% in F1 and 44.22% in Recall for the version without it.

6.3 ALFWorld Text World Interaction

Figure 3c presents the results for ALFWorld text world interaction. ALFChat with 3 agents demonstrated the best performance, achieving an average success ratio of 65% (69% in the original paper) and a best-of-3 ratio of 79% (77% in original paper). ALFChat with 2 agents showed lower performance, with an average success ratio of 51% (54% in the original paper) and a best-of-3 ratio of 68% (63% in the original paper).



(a) Results on SelectBenchmark (GPT-3.5 backbone)



(b) Results on SelectBenchmark (GPT-4o backbone)

Figure 4 Performance of the overarching agent on SelectBenchmark: (a) with GPT-3.5 and (b) with GPT-4o

6.4 Multi-agent Coding

Ratio per dataset	Original	Reproduced
tsp	4.88x(1.71)	4.65x(1.58)
coffee	3.38x(0.86)	3.5x(0.90)
diet	3.03x(0.31)	3.02x(0.31)

Table 1 Time-saving ratio

Table 1 shows the saving ratio of time for different tasks using the AutoGen framework. The mean and standard deviations (indicated in parentheses) of the saving time ratio are compared for tsp, coffee and diet tasks from the datasets of OptiGuide. It is worth noting that the capability of ChatGPT is at present not the same as when the AutoGen paper was introduced initially. Therefore, we could notice slight differences in the results. However, it shows the strengths of AutoGen for reducing the cost of time for users.

6.5 Dynamic Group Chat

Model	Two Agent	Group Chat
GPT-3.5-Turbo	6	8
GPT-4	9	7

Table 2 Number of successes on the 12 tasks (higher the better)

Table 2 shows some differences in performance compared to the original results. For GPT-3.5-Turbo, both Two Agent and Group Chat performances are lower in the recreation (6 vs 8 for Two Agent, 8 vs 9 for Group Chat). For GPT-4, the Two Agent performance remains consistent at 9 successes, but the Group Chat performance is notably lower in the recreation (7 vs 11 successes).

6.6 MiniWeb++ Web Interaction

Figure 3d shows the performance on MiniWeb++ for 23 out of 47 tasks. The graph compares the success ratio (%) between the original paper and the recreated results using gpt-3.5-turbo-16k. The average success ratio for the original results is 51.74%, while the recreated results show an average success ratio of 56.52%.

6.7 SelectBenchmark

Figure 4 presents the performance of the overarching agent on SelectBenchmark using GPT-3.5 and GPT-4 backbones. For GPT-3.5, the average success rate across the tasks is 55%, average called correct agent rate is 98.3% and average called only correct agent rate is 41.7%. With GPT-4o, the average success rate is 65%, average called correct agent rate is 96% and average called only correct agent rate is 57.5%. The GPT-4o backbone shows higher average success rate and average called only correct agent rate, while the average called correct agent rate remains similarly high for both backbones.

7 Findings

We can now answer the questions defined in section 1 below:

RQ1 Are the experiments conducted by the AutoGen authors reproducible?

Yes, while the results of individual experiments do not necessarily correspond one to one to the results of the original paper, the general observed trends are reproducible. Namely, AutoGen provides a noticeable boost in the quality of answers and provides a controllable environment for answering the questions.

RQ2 How effectively can an overarching agent, implemented using the AutoGen framework, delegate tasks to specialized sub-agents and solve complex problems across diverse domains?

The results of SelectBenchmark demonstrated the capabilities of the overarching agent to delegate the task to the appropriate agent and come up with a correct response most of the time, even if this response involved a long communication chain.

RQ3 What are the possible scenarios and tasks that could benefit from a multi-agent approach?

In our research, we saw that AutoGen provides benefits for further scenarios, such as browsing, generating images or parsing PDF documents. Without a doubt the multi-agent approach is applicable for other complex question answering scenarios and can provide improvements over the traditional single-agent approach.

RQ4 What are the key considerations in designing user interfaces to facilitate the creation and utilization of multi-agent systems?

To answer the question of facilitating user interface experience, we implemented demos for the experiments mentioned in this report using Python's Streamlit library. The code for these demos is published on GitLab (pro, 2024) and deployed on Oracle Cloud.

Limitations We reproduced the experiments found in the original paper and introduced additional scenarios where the AutoGen multi-agent framework can show its strengths. As shown in these experiments, multi-agent LLMs provide certain benefits, such as a more controlled environment, incremental improvements, and advancements in answer quality. At the same time, these benefits need to be taken with a grain of salt, as the multi-agent environment also introduces an increase in computation cost and processing times at times, as each request now requires several responses. It is also worth noting that based on our SelectBenchmark experiment results the precision of agent selection and reducing unnecessary invocations of multiple agents for single tasks remain areas for improvement. Furthermore, the unpredictability of LLMs can lead to accumulating errors, as the agents base their context on the answers of other agents.

8 Conclusion

In this study, we explored the benefits provided by a multi-agent approach to LLMs. As showcased by both the reproduced findings of the original paper and our experiments, it provides a marginal improvement in both answer quality and fidelity, but also comes with drawbacks in price and computational resources required to generate an answer. We further introduced an overarching agent aimed to solve a broader set of tasks and demonstrated its capabilities to select the right agent on our proposed dataset SelectBenchmark. Last but not least, we explored the applications of Streamlit to create a pleasant user experience for multi-agent framework and published our code on GitLab.

For future development of multi-agent LLMs, several avenues of future research could be explored. One area of investigation could focus on whether the overarching agent would benefit from incorporating more specialized agents, and whether its ability to delegate tasks might be impacted by an increase in the number of participants. Furthermore, the process of selecting the appropriate agent for delegation could be explored and enhanced, for example via introducing a hierarchical communication structure, similar to the real-world org-charts.

Another potential direction involves examining whether specialized agents would benefit from additional fine-tuning, and whether agents dedicated to specific tasks should be less general in their approach. Finally, it would be valuable to compare the overarching agent implemented in AutoGen with similar frameworks to assess its relative strengths and potential areas for improvement.

References

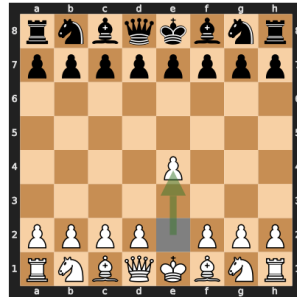
- Multi-tool LLM agents - AutoGen. <https://gitlab.lrz.de/social-computing-research-group/nlp-lab-course/nlp-lab-course-ss2024/multi-tool-llm-agents-autogen>, 2024. Accessed: 2024-08-13.
- Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Giani-nazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, Piotr Nyczyk, and Torsten Hoeffler.

- Graph of thoughts: Solving elaborate problems with large language models. In *Proceedings of AAAI*, pages 17682–17690, 2024.
- Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. In *Proceedings of NeurIPS*, 2020.
- Marc-Alexandre Côté, Ákos Kádár, Xingdi Yuan, Ben Kybartas, Tavian Barnes, Emery Fine, James Moore, Matthew Hausknecht, Layla El Asri, Mahmoud Adada, et al. TextWorld: A learning environment for text-based games. In *Computer Games Workshop at IJCAI*, 2018.
- Victor Dibia, Gagan Bansal, Jingya Chen, Suff Syed, Adam Fourney, Erkang Zhu, Chi Wang, and Saleema Amershi. Introducing AutoGen Studio: A low-code interface for building multi-agent workflows. Microsoft Research Blog, 2024. URL <https://www.microsoft.com/en-us/research/blog/introducing-autogen-studio-a-low-code-interface-for-building-multi-agent-workflows/>. Accessed: 2024-08-11.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *Proceedings of NeurIPS*, 2021.
- Geunwoo Kim, Pierre Baldi, and Stephen McAleer. Language models can solve computer tasks. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, 2023.
- Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur P. Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, et al. Natural questions: a benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7: 452–466, 2019.
- Beibin Li, Konstantina Mellou, Bo Zhang, Jeevan Pathuri, and Ishai Menache. Large language models for supply chain optimization. *arXiv preprint arXiv:2307.03875*, 2023a.
- Guohao Li, Hasan Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. CAMEL: Communicative agents for "mind" exploration of large language model society. In *Proceedings of NeurIPS*, 2023b.
- Evan Zheran Liu, Kelvin Guu, Panupong Pasupat, Tianlin Shi, and Percy Liang. Reinforcement learning on web interfaces using workflow-guided exploration. In *Proceedings of ICLR*, 2018.
- OpenAI. DALL·E 3. <https://openai.com/dall-e-3>, 2023. Accessed: 2024-08-14.
- Jon Saad-Falcon, Joe Barrow, Alexa Siu, Ani Nenkova, David Seunghyun Yoon, Ryan A. Rossi, and Franck Dernoncourt. PDFTriage: Question answering over long, structured documents. *arXiv preprint arXiv:2309.08872*, 2023.
- Tianlin Shi, Andrej Karpathy, Linxi Fan, Jonathan Hernandez, and Percy Liang. World of bits: An open-domain platform for web-based agents. In *Proceedings of ICML*, pages 3135–3144, 2017.
- Mohit Shridhar, Jesse Thomason, Daniel Gordon, Yonatan Bisk, Winson Han, Roozbeh Mottaghi, Luke Zettlemoyer, and Dieter Fox. ALFRED: A benchmark for interpreting grounded instructions for everyday tasks. In *Proceedings of CVPR*, pages 10740–10749, 2020.
- Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. ALFWorld: Aligning text and embodied environments for interactive learning. In *Proceedings of ICLR*, 2021.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Proceedings of NeurIPS*, 2022.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation framework. In *arXiv preprint arXiv:2308.08155*, 2023.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Proceedings of NeurIPS*, 2023.

Shaokun Zhang, Jieyu Zhang, Jiale Liu, Linxin Song, Chi Wang, Ranjay Krishna, and Qingyun Wu. Offline training of language model agents with functions as learnable weights. In *Proceedings of ICML*, 2024.

Appendix A



Player black (to Player white):

Alright, let's get this game rolling!

1... e7e5

Your move! Ready for a fun match?



Player white (to Player black):

Great! We're off to a classic start.

2. Ng1f3

Your move! This is going to be exciting!

Figure 5 Visualization of the first move of each player in a sample chess interaction.

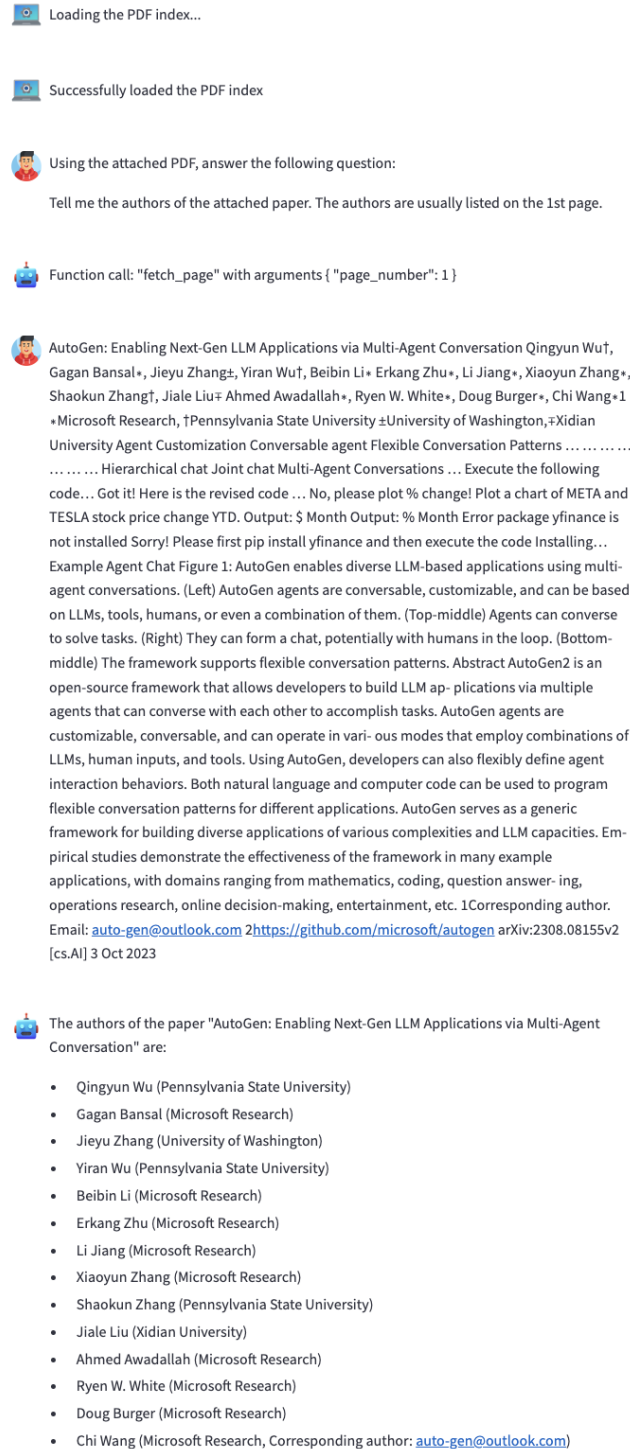


Figure 6 Chat visualization of PDFTriage agent on the AutoGen paper.



Answer the following question:

Generate a close-up image of a red apple



PROMPT: Here is the updated prompt!

Generate a high-resolution, close-up image of a vibrant red apple with a glossy surface. The apple should have a small green leaf attached to its stem, and the background should be softly blurred to emphasize the apple's details and color.



I generated an image with the prompt: High-resolution, close-up image, vibrant red apple, glossy surface, small green leaf on stem, softly blurred background.<image>



Figure 7 Conversation visualization of DALL-E agent.



Answer the following question:

Describe this landscape. What time of day do you think it is?

You can use the attached image (<image_url>) to get context.



The landscape in the image is a picturesque scene featuring a serene lake surrounded by mountains. The lake is calm and reflects the clear blue sky above. The mountains in the background are partially covered with snow, indicating a higher altitude. In the foreground, there is a quaint village with charming houses, a church with a red steeple, and lush green vegetation. The area appears to be well-maintained and vibrant with greenery.

The sky is clear and bright blue, suggesting that it is a sunny day. The shadows cast by the buildings and trees indicate that the sun is relatively high in the sky, which suggests that it is likely late morning or early afternoon.



It seems that the image_explainer has provided a detailed description of the landscape and suggested that it is likely late morning or early afternoon based on the lighting and shadows. If you have any further questions or need additional assistance, feel free to ask!

Figure 8 Chat visualization showcasing vision agent's image processing capabilities on a landscape image.



Answer the following question:

Give me the best three platforms for sharing my code with others



Function call: "search" with arguments {"query": "best platforms for sharing code 2023"}

Function call: "search" with arguments {"query": "top code sharing platforms"}



Search result #0 Title: Free recommended code sharing sites? - Python Help - Discussions ...

Snippet: Feb 15, 2024 ... September 21, 2023. Home · Categories · Guidelines · Terms of Service · Privacy Policy. Powered by Discourse, best viewed with JavaScript ... Link:

<https://discuss.python.org/t/free-recommended-code-sharing-sites/46024>



Search result #9 Title: The Journey to Sharing America's Code – Digital.gov Snippet: Jun 27, 2017 ... Code.gov the nation's leading platform for sharing and improving government software. This is where you come in. Teamwork Makes the Code Work. Link:

<https://digital.gov/2017/06/27/the-journey-to-sharing-americas-code/>



Based on the search results, here are three of the best platforms for sharing your code with others:

1. GitHub Gist:

- GitHub Gist is a simple way to share code snippets and notes with others. It integrates seamlessly with GitHub, making it easy to share and collaborate on code.

2. CodePen:

- CodePen is a social development environment for front-end designers and developers. It allows you to write HTML, CSS, and JavaScript code directly in the browser and see the results in real-time. It's great for sharing and showcasing your work.

3. JSFiddle:

- JSFiddle is an online IDE that allows you to write and share HTML, CSS, and JavaScript code. It provides a simple interface to test and debug code snippets, making it a popular choice for web developers.

These platforms are widely used and offer various features to facilitate code sharing and collaboration.

Figure 9 Visualization of web search and retrieval conversation for interactive browsing agent.

Task name: Simple Object Generation	
Input:	Generate a close-up shot of a red apple
Success Criteria:	"Image contains a clearly recognizable red apple", "Image is of high quality and resolution", "Image is a close-up shot"
Expected Agent:	Dalle

Table 3 Example task, where the *Overarching* agent should select *DALL-E* agent. In this case the performance is evaluated by generating a textual description of the new image DALL-E created using GPT4-V and comparing it to our success criteria.

Task name: Landscape Description	
Input:	Describe this landscape. What time of day do you think it is?
Path URL:	https://images.unsplash.com/photo-1472214103451-9374bd1c798e
Success Criteria:	"Provides a detailed description of the landscape (grass, stones, mountains, sky)", "Makes a reasonable estimate of the time of day (Sunset, Dawn, Noon, Sunrise, etc.)"
Expected Agent:	Image Explainer

Table 4 Example task, where the *Overarching* agent should select *Image Explainer* agent.

Task name: Fibonacci Sequence	
Input:	Write a function that generates the first n numbers of the Fibonacci sequence. Return the result for n = 10
Success Criteria:	"The result is [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]"
Expected Agent:	Coding

Table 5 Example task, where the *Overarching* agent should select *Coding* agent.

Task name: Authors List	
Input:	List the authors.
Path URL:	https://arxiv.org/pdf/1803.02155.pdf
Success Criteria:	"Authors: Peter Shaw, Jakob Uszkoreit, Ashish Vaswani"
Expected Agent:	PDF Parser

Table 6 Example task, where the *Overarching* agent should select *PDF Parser* agent.

Task name: Euro 2024 Match Schedule	
Input:	When did Spain play against Germany in the Euros 2024?
Success Criteria:	""Correctly identifies the date as 05.07.2024", "Confirms that this is a match between Spain and Germany in Euro 2024"
Expected Agent:	Web Retriever

Table 7 Example task, where the *Overarching* agent should select *Web Retriever* agent.

Task name: Mixed Operations	
Input:	What is $(10 + 5) * 2 - 8$?
Expected Result:	22
Success Criteria:	"Correct result (22) is returned"
Expected Agent:	Calculator

Table 8 Example task, where the *Overarching* agent should select *Calculator* agent.